

Session_13 activities worksheet

Online handouts: `gaussian_random.cpp`, `random_walk.cpp`, and other listings.

Today we'll play some games with the GSL random number generators.

Your goals for today:

- Generate some random walks and verify their properties.
- Try out primitive Monte Carlo integration.
- Take a look at an alternative version of the random walk code using classes.

Please work in groups in the breakout rooms.

A) Random Number Generation

The program `gaussian_random.cpp` calls GSL routines to generate both uniformly distributed and gaussian distributed numbers.

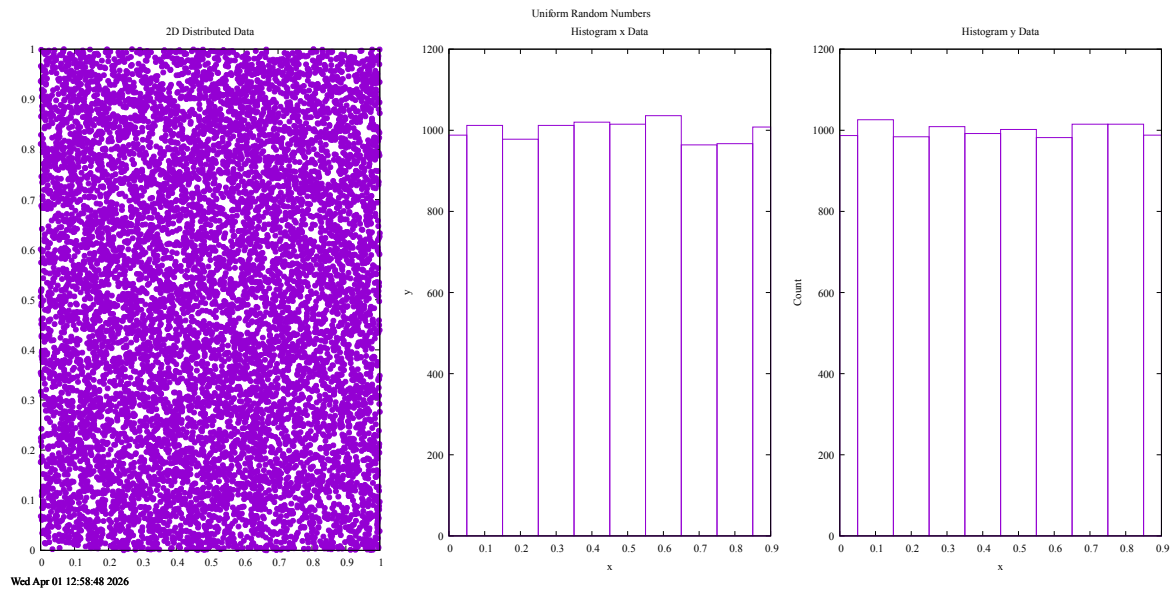
1. Look at the `gaussian_random.cpp` code (there is a printout) and identify where the random number generators are allocated, seeded, and called. *If you were creating a `RandomNumber` class, what statement(s) would you put in the constructor and destructor? What private variables would you define?*

→ In the constructor, I would probably want the constructor to take in the seed as a parameter and would have functions to generate N random numbers, and it would also allocate the rng. I would maybe also want the lower and upper bounds and the standard deviation as variables, and the destructor should help free the allocated memory. The seed should also be a private variable.

Compile and link the code (use `make_gaussian_random`) then generate pairs of uniformly and gaussian distributed numbers in `random_numbers.dat`.

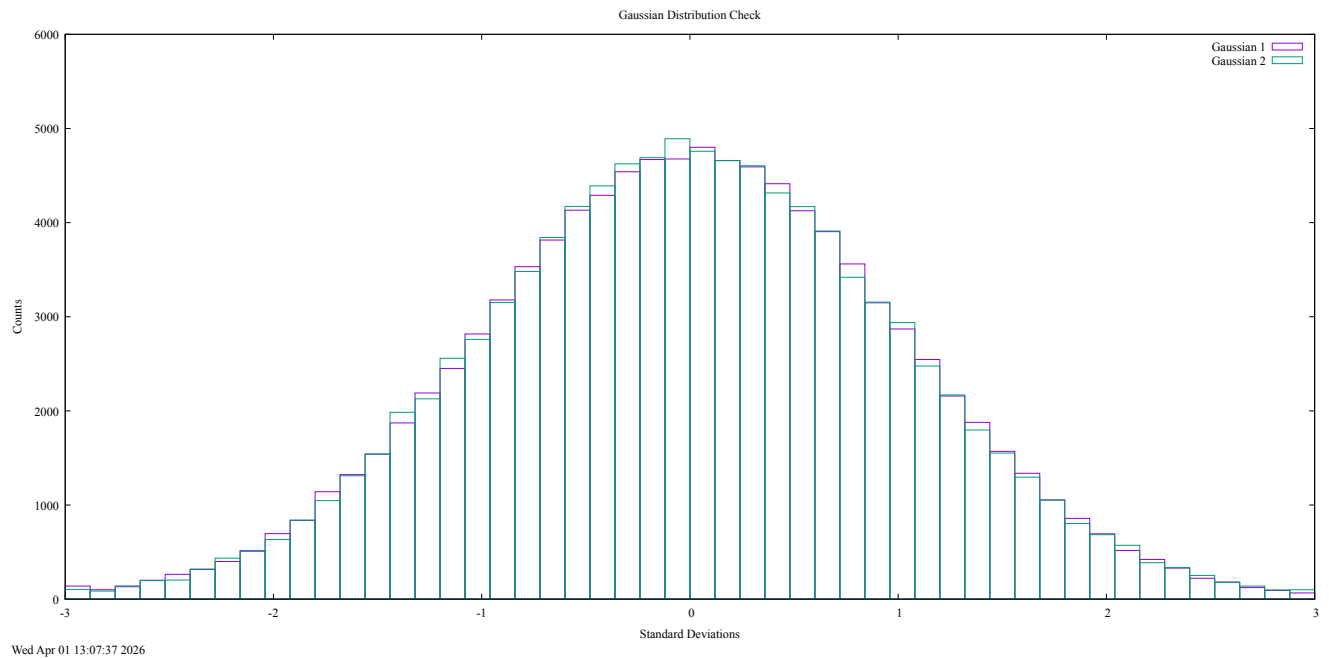
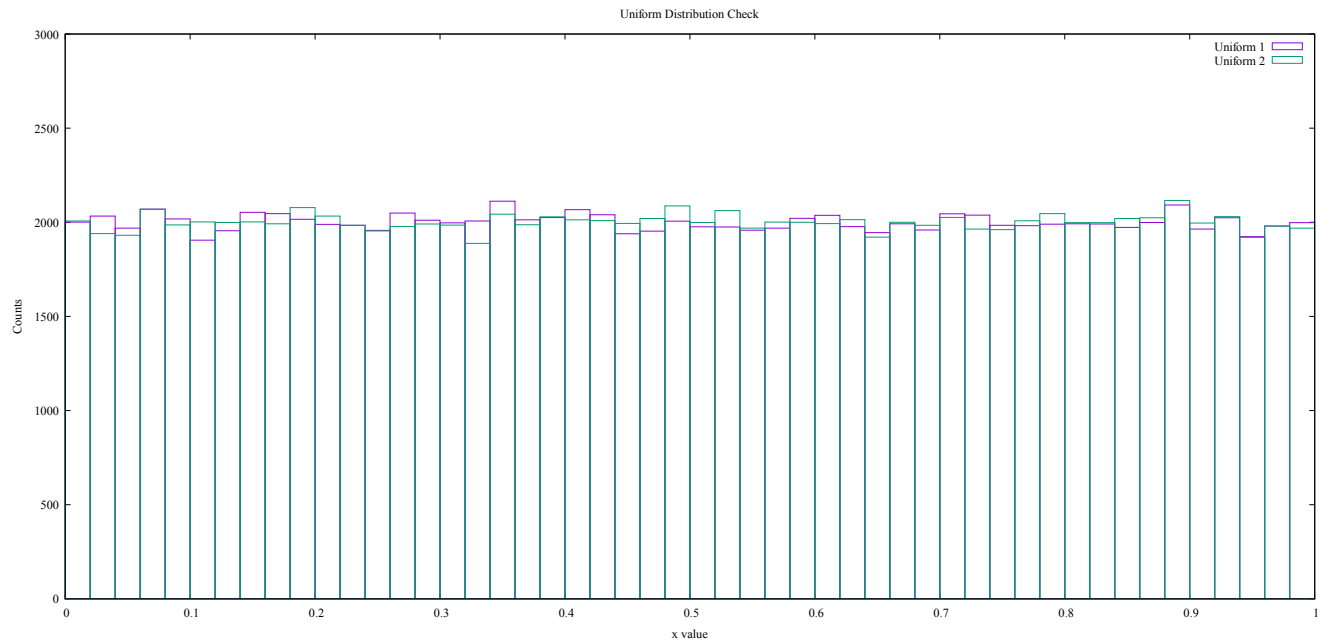
2. Devise and carry out a way to use gnuplot to roughly check that the random numbers are uniformly distributed. [Hint: Read the notes. Your eye is a good judge of nonuniformity in two dimensions.] *What did you do?*

→ You can make a plot of the x and y data and see that the data fills the grid and shows no clear pattern. You can also histogram the x and y data and see if the distribution is roughly uniform.

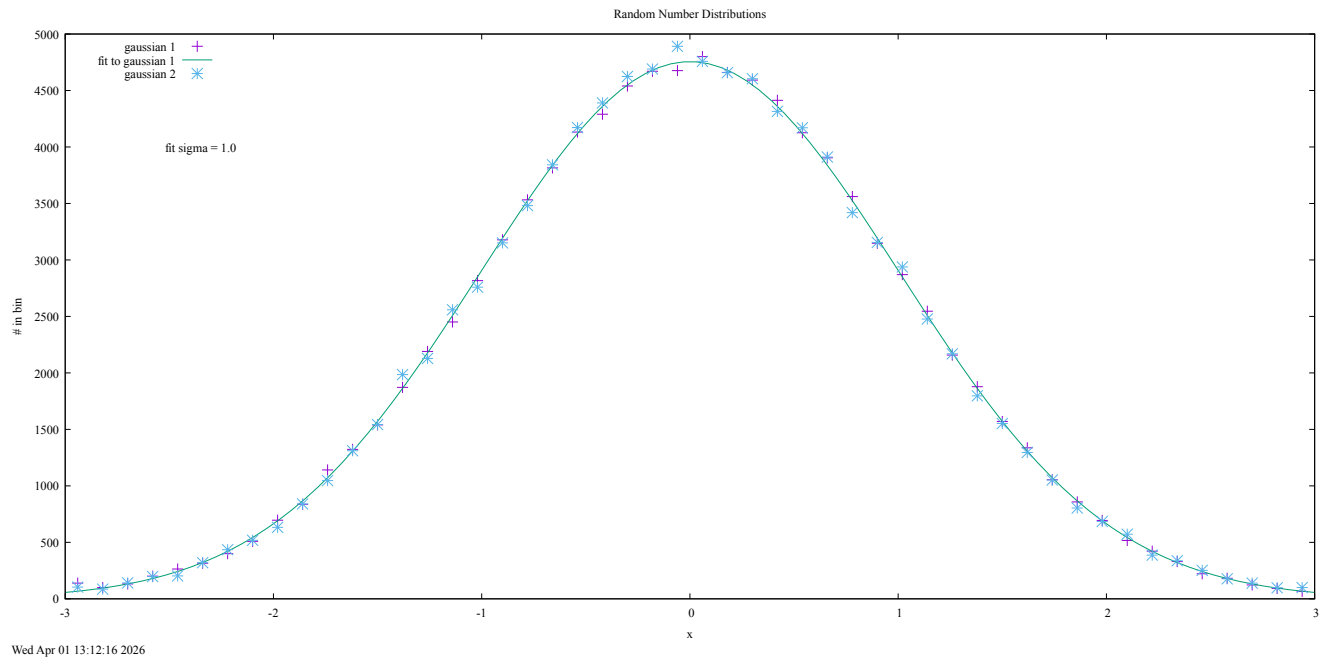


3. You can check the distributions more quantitatively by making histograms of the random numbers. Think about how you would do that. Then take a look at `gaussian_random_new.cpp`, which has added crude histogramming (as well as automatic seeding). Use the makefile to compile and run with about 100,000 points. Look at `random_histogram.dat`. *Use gnuplot to plot appropriate columns (with appropriate ranges of y) to check the uniform and gaussian distributions. Do they look random? In what way?*

→ The histogram plots are below. They are random, just in different ways. The uniform distribution is random by having a uniform assortment of values (as seen in the histogram), but we also see fluctuations in the height of each bin due to the randomness. For the Gaussian, it appears ordered due to the bell curve, but it is still random, as seen by the fluctuation in the boxes (it's not an exact Gaussian), and we see how the two Gaussians vary although both being generated by the same method. The plots are below:



4. Run `gaussian_random_new.plt` to plot and fit the gaussian distributions with gnuplot. Try 1,000,000 points and 10,000 points. *Do you reproduce the parameters of the gaussian distribution? Attach a plot.* (You may need to set `b` to a reasonable starting point such as the approximate peak height to get a useful fit.)
- The plot is below, and we do recover the parameters of the Gaussian distribution!



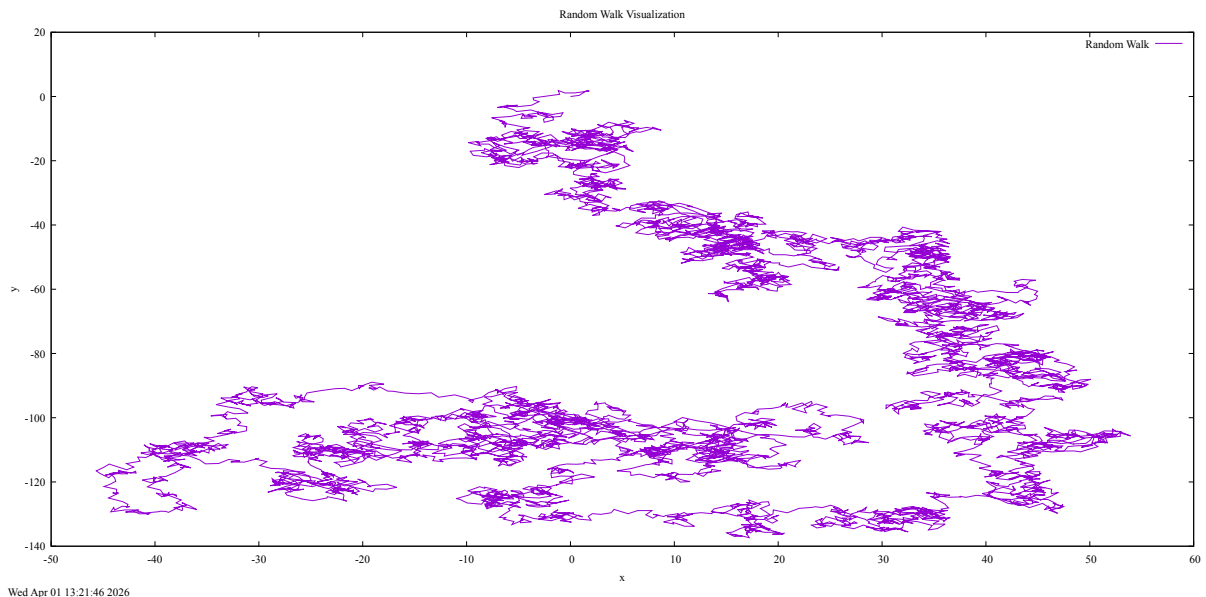
5. [Extra (do last)] Run `rolling_dice.nb` to see how to do random numbers in Mathematica.

→ Done!

B) Random Walking

We'll generate random walks in two dimensions using method 2 from the list in Section b of the Session_13 notes. In particular we'll start at the origin: $(x,y) = (0,0)$ and for each step select Δ_x at random in the range $[-\sqrt{2}, \sqrt{2}]$ and Δ_y in the same range. So positive and negative steps in each direction are equally likely. The code `random_walk.cpp` implements this plan.

1. *What is the rms step length?* (Note: this is tricky!)
 → The RMS step length would be the $\sqrt{\langle L^2 \rangle}$ or the $\sqrt{\langle \Delta_x^2 + \Delta_y^2 \rangle}$. Thus, for our code with uniform distribution between $[-\sqrt{2}, \sqrt{2}]$, then $\langle \Delta_x^2 \rangle = (b-a)^2/12 = 2/3$. Hence the RMS step length is $\sqrt{2/3 + 2/3} = 1.15$.
2. Look at the `random_walk.cpp` code and identify where the random number generator is allocated, seeded, and called. Compile and link the code (use `make_random_walk`) and generate a random walk of 6400 steps.
3. Plot the random walk (stored in "`random_walk.dat`") using `gnuplot` (use "`with lines`" to connect the points). Repeat a couple of times to get some intuition for what the walks look like.
 → An example plot is below:



4. Check (using an editor) for the endpoints of a few walks. *Roughly how does a typical distance R from the origin scale with N ? (Can you reproduce the derivation from the notes of how the average of R scales with N ?)*

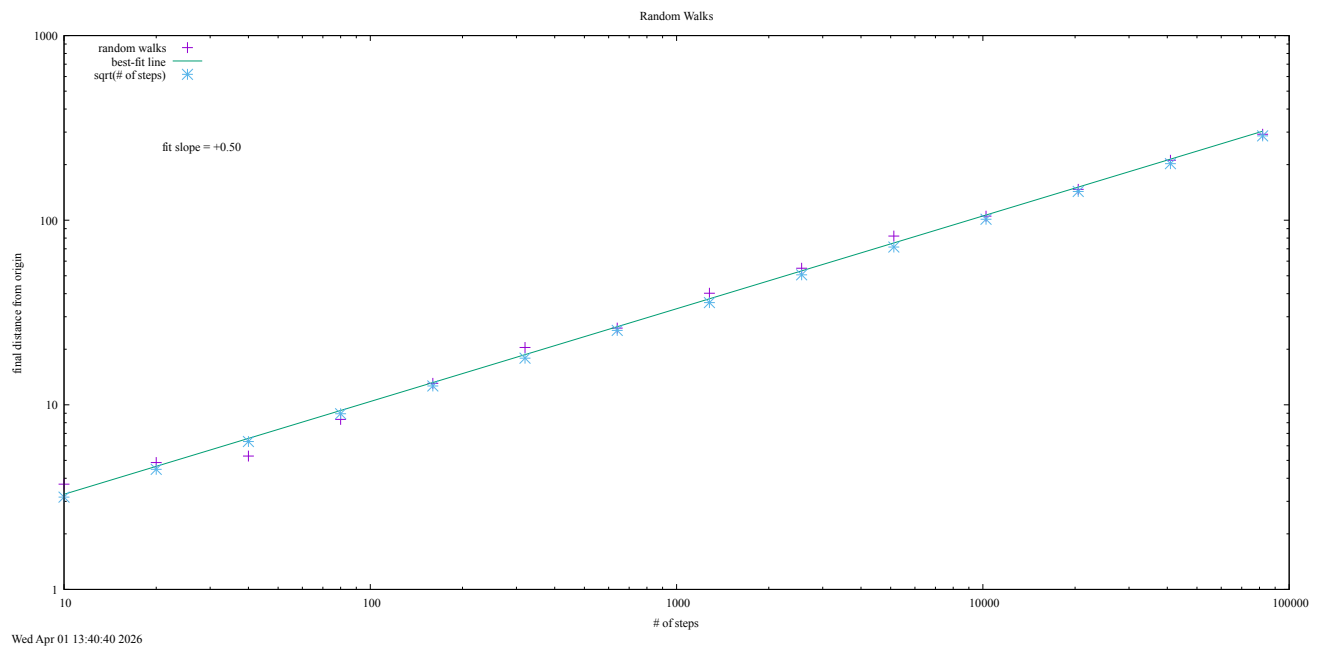
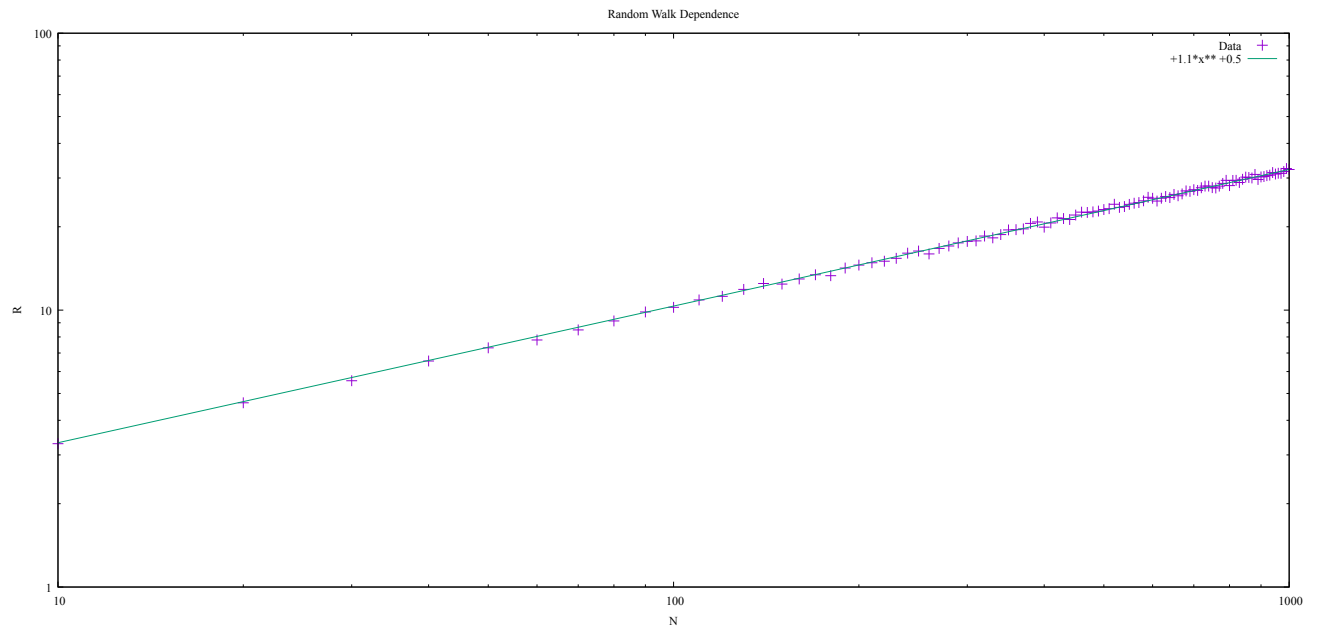
→ For $N=1000$, we get R roughly equals 50. For $N=10,000$ we get R roughly equals 176. For $N=100,000$ we get R roughly equals 565. To reproduce the derivation in the notes, as N increases by a factor of 10, R should increase by a factor of about 3 (since $R \propto \sqrt{N}$). We see this is relatively accurate, as $50 \cdot 3 = 150$ and $176 \cdot 3 = 528$, both of which are close to their next R value.

5. Now we'll study more systematically how the final distance from the origin $R = \sqrt{x_{\text{final}}^2 + y_{\text{final}}^2}$ scales with the number of steps N . Note that now we don't need to save anything from a run except the value of R . The value of R will fluctuate from run to run, so for each N we want to average over a number of trials. *How many trials should you use?*

→ Since the standard error tends to go as $1/\sqrt{N}$, to balance precision and computational time, it is best to use 1,000-10,000 trials, as over that would take too long, and doing less like 100 trials would only allow for maybe 10% precision.

Edit the code to make multiple runs for each value of N and takes the average of R . *Make (and attach) an appropriate plot that reveals the dependence of R on N .* [The code `random_walk_length.cpp` and plot file `random_walk_length.plt` implement this task. Try it yourself before looking at those.] *Does it agree with expectations?*

→ Editing the code and plotting on a logscale and fitting gets us the expected dependence of R on N (an exponent of 0.5 or $R \propto \sqrt{N}$). This agrees with the result of `random_walk_length.cpp` as well. Both plots are below:



C) Monte Carlo Integration: Uniform and Gaussian Sampling

Your goal is to estimate the D-dimensional integral of

$$(x_1 + x_2 + \dots + x_D)^2 \frac{1}{(2\pi \sigma^2)^{D/2}} \exp(-(x_1^2 + x_2^2 + \dots + x_D^2) / (2 \sigma^2))$$

where each of the variables ranges from $-\infty$ to $+\infty$. The exact answer is $D \cdot \sigma^2$. [Note that the integral without the squared sum in front is normalized to be one.]

The basic Monte Carlo integration method is described in Section d of the Session_13 notes. In particular, equations (13.15) and (13.16) show that the integral is given approximately by the range(s)

times the average of the function evaluated at N random vectors. (So for a 5-dimensional integral, each vector is a set of 5 random numbers $\{x_1, x_2, x_3, x_4, x_5\}$.)

1. Look at `mc_integration.cpp` to see how this is implemented for our test integral. Because the integral has infinite limits, we approximate it with finite lower and upper limits. *How would you choose these?*

→ You would choose these using the standard deviation of a Gaussian distribution. At 3 sigma, we enclose about 99.7% of the total area, and even more for higher sigma. It appears that's what we do in the code, choosing a 5 sigma range.

2. The dimension is initially set to $D=1$ (called `dim` in the code). Compile it with `make_mc_integration` and run it several times. After each run, use `mc_integration.plt` to make a fitted plot of the error. *What do you observe?*

→ I observe that the values are changing with each run, thus changing the fit slope, but it tends to stay around -0.5. We also see that the absolute error is always in general decreasing when you increase the number of integration vectors.

3. Next try changing the dimension to 3 and then to 10, repeating the last part. *What do you observe? Is it consistent with the notes?*

→ We observe that for higher dimensions, the slope values seem to remain more constant with runs, but also the slope is decreasing, getting down to -0.2 to -0.4 for 10 dimensions. The notes expect a slope of -0.5, which is close to what we get with the lower dimensions, but the notes also mentions that among higher dimensions we expect $N^{-4/D}$, which does match the -0.4 slopes we get for 10 dimensions, but it also doesn't really explain getting down to -0.2.

4. If you have time, modify the code to apply equations (13.34) and (13.35), where we identify $w(x)$ as the normalized gaussian part of the integral and use the GSL routine for generating gaussian-distributed random numbers (from `gaussian_random.cpp`). [If you are short of time, use `mc_integration_new.cpp`.] *What should the integrand function return in this case?*

→ The integrand function should return the $f(x)$ part of the function, or $x_sum * x_sum$ in the code.

Repeat the analysis in different dimensions. *Why are the results better than for uniform sampling? Can you do $D=100$?*

→ The results are better than uniform sampling because it “directs” all the random samples to the regions where the integrand is actually non-zero. Because of this, we can actually perform $D=100$, because the method stays stable, whereas uniform sampling would likely fail with $D=100$.

D) Monte Carlo Integration: GSL Routines

Run a test program to do a simple D-dimensional integral as in the last section but with the Vegas and Miser programs.

1. Take a look at the program `gsl_monte_carlo_test.cpp` while also looking at Monte Carlo integration in the online GSL library.
2. The initial integral is not a great test. After compiling and running the program, change the integrand to something more interesting (use your imagination!). Don't worry about knowing the exact answer; compare the results from the different routines. *What do you find?*

→ With 10 dimensions and a more complicated integrand, we see that Plain and Miser provide only rough estimates with very high statistical noise, while Vegas outperforms them. Instead of the simple integrand, I used a varied normalized Gaussian of form $x^2 \cdot \exp(-x^2)$.

E) C++ Class for a Random Walk

The random walk code `random_walk.cpp` is basically written as a C code with C++ input and output. Here we reimplement the code as a C++ class.

1. In the RandomWalk directory, compile and link `RandomWalk_test` (using `make_RandomWalk_class_test`). Run it to generate "RandomWalk_test.dat", which you should plot with gnuplot to verify that the output looks the same as from `random_walk.cpp`.
2. Compare the old and new code (you have printouts of each). Discuss with your partner the advantages (and any disadvantages) of the definition of RandomWalk as a class. List some.

→ Some advantages of having a class: better readability, understandability, and scalability, and everything is encapsulated better. Some disadvantages: probably increasing overhead, lots more bloating with setter and getter function, and likely includes hidden logic that's not always obvious.

3. An advantage of programming with classes is the ease of extending or generalizing a code. List two ways to extend the class definition.
→ One way to extend the class definition is through inheritance, like creating a subclass for a different kind of random walk. The other is extending by adding new functions, like a `getDistance()` function or alike.

4. As time permits, modify the code to do the following:
 - Extend the code to make available (with "get" functions) the x- and y-components of the last step (what are called `delta_x` and `delta_y` internally).
 - Allow for the upper and lower limits of the step size to be initialized by the user. (And you still want to be able to use the current version that doesn't require these.) [Hint: Can you have more than one constructor?]

- *How would you allow for the random number generator to be changed? Implement it!*
→ To allow for the RNG to be changed, we need to move it out of the constructor and use a `gsl_ptr`.
 - [For the experts!] *How would you dynamically create and destroy random walks? [Hint: new and delete]*
→ For dynamic creation, you would create a pointer to a `RandomWalk` and use the `new` keyword to dynamically allocate the object. Then you can access its members using “->” and delete it with the `delete` keyword or by setting it to `nullptr`.
-

F) Debugging Revisited: Factorial Program

Time for more debugging practice. This time we have a program called `factorial_debug.cpp` that recursively calculates factorials and stores them in arrays.

1. Try to compile using `make factorial_debug.cpp`. There might be an error about a "variable-sized" array. If so, the problem is that we can't declare `stored_factorials` when `n_max` is not a constant. Solution? Add `"const"` before `"int n_max = 5;"`. Try it!
 2. Ok, now it should compile but it dies with a segmentation fault. Use GDB, including a traceback, to identify and fix the problem. *What was wrong?*
→ The function `factorial` introduces an infinite loop. The return statement should be `n*factorial(n-1)`, not `n*factorial(n)`. The crashing on the OSC is likely associated with too much RAM usage or equivalent.
 3. Now there is either a new segmentation fault or wrong output (depending on your computer). *What is the problem?*
→ The output for the `stored_factorials` is not correct, since both indices are out of range (-1 and 5) for an array of length 5. If we want the end values, we can use 0 and `n_max-1`.
 4. With that fixed, you can get to the part of the code where an array is *dynamically* generated with `"new"`. *What statement would you use to create an array called "x_array" of doubles with integer length "x_length"?*
→ Based on the way the syntax is in the code, a new array like that would be declared using the syntax: `double *x_array = new double[x_length];`.
 5. If you put in too large an integer, bad output happens. *How could you store factorials differently to avoid this problem?*
→ You could store factorials with larger built-in types that could handle the large numbers, but that would likely cap out around 20! with unsigned long long or maybe larger with a double or long double. We could also motivate the use of natural logarithms, where something like 20! Which is roughly $2e18$, can be stored as $\ln(20!) = 42.33$. This is a smaller number which we can revert later to get the larger value, allowing us to calculate larger factorials as well. There are also other ways to get factorials that we could use as well.
-

G) Reflection

Write down which of the six learning goals on the syllabus this worksheet addressed for you and how:

→ This worksheet helped me learn more of the limits to computational solutions to physics problems, like how dimensionalities impact results alongside a size N . I also engrained more tools of computational physics like GDB debugging or understanding random numbers and walks.